Business-Actors as base components of complex and distributed software applications

Florian Strecker Reinhard Gniza Thomas Hollosy Florian Schmatzer
actnconnect
Frankenstr. 152
90461 Nuremberg, Germany
+49 911 3606108-0
florian.strecker@... reinhard.gniza@... thomas.hollosy@... florian.schmatzer@...
…actnconnect.com

ABSTRACT

Subject-oriented Business Process Management (S-BPM) is an emerging approach focusing on adjusted interaction and individual behavior of stakeholders in business operation. Unfortunately, current implementations & tools lead to issues in most industry-related S-BPM-projects. This contribution reveals how the concept of Business-Actors reduces S-BPM to it's underlying core and enhances it at the same time in some fields. Thus, Business-Actors are a method and technology being able to overcome problems of classical S-BPM-approaches.

CCS Concepts

• *Computing methodologies~Multi-agent systems*

• *Applied computing~Business process management systems*

• *Applied computing~Service-oriented architectures*

• **Applied computing~Business process modeling**

• **Applied computing~Cross-organizational business processes**

• **Applied computing~Business-IT alignment**

• ***Software and its engineering~Cooperating communicating processes***

• ***Software and its engineering~State systems***

Keywords

Business-Actors, PASS, Subject-oriented BPM, Distributed Software Applications, Complexity, Microservices.

S-BPM '16, April 07 - 08, 2016, Erlangen, Germany

ACM 978-1-4503-4071-7/16/04…$15.00
DOI: http://dx.doi.org/10.1145/2882879.2882887

1 INTRODUCTION

As organizations need to act flexible in the continuously changing landscape of the digital economy, their process work is increasingly driven by valued interactions among stakeholders. Subject-oriented Business Process Management (S-BPM) is an emerging approach that adresses exactly these drivers by focusing on the communication between process participants [1][2][3].

During the last decade, the first software tools using S-BPM as underlying method, emerged in the market. Unfortunately, these tools are not able to adress some of the real-world problems arising in industry-related projects. Above all, the main issues are the tight coupling of S-BPM-subjects and the absence of user-interface elicitation techniques matching the expectations of business users [2][6].

In this contribution we present the approach of using so called Business-Actors (BA) as base components of complex and distributed software applications. We show why this approach can overcome 'classical' problems of S-BPM, BPMN and other methods. Furthermore, we emphasize on the generation of flexible, adaptable (and therefore resilient) networked software architectures, which are facilitated by the Business-Actor-approach.

2 PRACTICAL PROBLEMS IN STANDARD-S-BPM-TOOLS

This section depicts the most common issues that arise in industry-related S-BPM-projects for business process automation. The assumptions are based on our many longtime practical S-BPM project experiences.

2.1 Tight Coupling between Subjects

The S-BPM standard process elicitation approach follows most cases a structure of first defining subjects participating in a process. The next step is to elicit or create communication channels between these subjects. These steps are done on the level of an subject-interaction diagram (SID). Only after these actions are done, the different subject-behaviours (state machines) are created and refined, whereby the communication is also refined if needed..

The industry-related S-BPM tools follow this standard approach and focus therefore on a tight coupling between the subjects drawn on the SID. Even as S-BPM as a method focuses on the communication between subjects as a key element and enabling mechanism for flexibility, current tools lack this kind of flexibility by coupling subjects together in a strict way (e.g. many

subjects in a single "process" file, primary key – foreign key constraints and relations in databases etc.

Thus, one of the main pros of S-BPM, the flexibility (and therefore agility) to swap one subject with another, that has the same communication interfaces, is sometimes lost entirely or gets expensive at least.

2.2 Missing User Interface Elicitation & Declaration

Over the years, we have conducted many process elicitation workshops and have gone through many process automations with endusers and managers.

In most S-BPM process elicitations so far, one focuses first on the creation of the SID (see above) and next on the creation of multiple state machines (subjects) and the data they use and swap via their communication.

The first user interfaces are generated automatically by the available S-BPM tools and shown to the end-users for process execution. This generation is solely based on the data fields provided during the elicitation and produces simple one- or two-column field input lists in most cases.

This approach follows a principle: Custom-programmed user interfaces (UI) are known for providing the perfect UI-structure for a specialized task. Workflow tools like S-BPM suites try to provide "good" UIs suitable for any task.

In our experience, end-users rather expect the perfect UIs than the good ones even if they know the intention of a workflow system of having "good" ones. Therefore, people often simply do not believe that these UIs (which are automatically created) are meant as the real end-product and not just for testing purposes. Additionally, current S-BPM tools do not offer many possibilities to customize the UIs created.

Thus, the lacking ability to customize also leads to a lack of elicitation of the best UI for the process at hand, because it would not make sense anyway because of missing tool-support.

To put it in a nutshell, the lack of real and extensive UI customization neither in elicitation phase nor in software creation phase of S-BPM projects is a major point of failure.

2.3 Unclear IT-Service & Rules Layers

The third major issue is the unclear layering of behaviour rules, business rules, data rules and integration of IT-services (like SOAP-services, RPC calls and other legacy services).

The S-BPM tools currently available on the market offer different possibilities to add these different building blocks to the S-BPM models.

The elements can be added on single states of the state machines, can even replace single steps of the state machines. They can be added on the data-level or, if complex data are used, on many different extension points on business-objects.

In the end, important information about services, rules etc. is distributed over many different elements and building blocks throughout the process. Thus "hiding" this information, forfeiting the advantage of transparency in S-BPM.

3 THE BUSINESS-ACTOR

To overcome the disadvantages depictured in section II we developed the concept of a Business-Actor. In this section we describe this concept in detail.

3.1 Building blocks of the Business-Actor

The Business-Actor comprises seven building blocks:

1. One (or more) state machine(s), with states referencing data from the data block and referencing actors for evaluation of data- and behavior-rules.
2. A data block, consisting of every data element, the Business-Actor is able to use (read, write) within its behavior.
3. Zero or more presentation-layers, referencing states from the state machines. Intended for storing information about graphical representation(s) of the Business-Actor.
4. Zero or more service-layers, providing service-mappings (data-input and –output, mapping to webservices (e.g. REST/ SOAP), RFC-interfaces, custom libraries, etc.) for each "do"-state of the state-machines. These service-layers are needed for Service-Actors (full automation of a Business-Actor).
5. Zero (in case of an automated Business-Actor, a.k.a. Service-Actor) or more abstract userinterface-definitions, each referencing data from the data block
6. A non-required communication-block containing communication-rules and –requirements regarding different communication-partners of the actor.
7. A metadata-block with further, human-readable information about the actor, owner, KPI's etc.

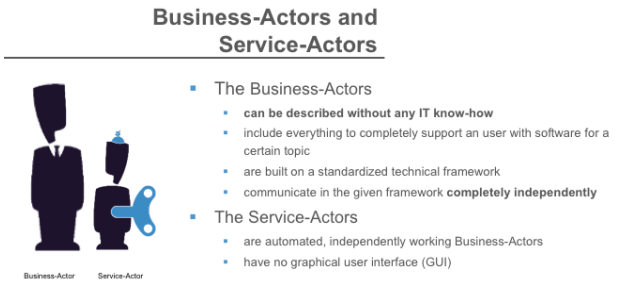


Figure 1. Business-Actors vs. Service-Actors.

These seven different building blocks contain all information needed for a Business- (or Service-)Actor to run self-sustaining and indepently from other software components:

The state machine(s) provide(s) information about the behavior of the actor, whereas the references to the data block indicate, which data are used at which point in time relative to the behaviour. In case of Service-Actors, the service-layers provide access to implementations of system-services, whereas the userinterface-definitions (in case of Business-Actors) provide access to all information needed to create userinterfaces for different input devices for human usage. Therefore, these elements could be compared to a fully-fledged computer program or a microservice. It can run independently of other programs, comprises its own business logic (i.e. behavior, sometimes in conjunction with custom service-implementations or –mappings) and can even be run distributed on different infrastructures. Each instanciation auf a Business-Actor can run on a different server, provided the model its based on is known there.

The communication parts (implicit within the state machine(s) and explicit within the communication block) and their impact on the concept are described in the next chapter.

The remaining blocks (presentation layers and metadata) have the purpose of adding information about graphical representations, KPI, SLA and ontology data ("Where can the Business-Actor being used?" "What is its purpose?" …) to the

definition of the Business-Actor. These items will not be explored in detail here.

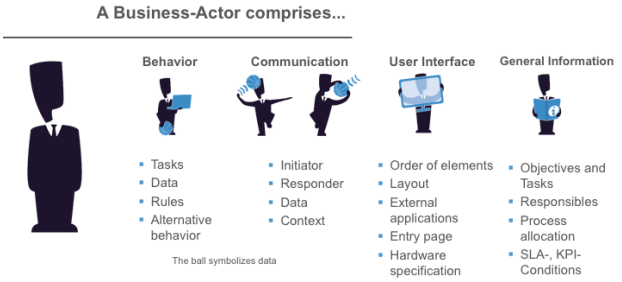


Figure 2. A Business-Actor in a nutshell.

3.2 Communication of the Business-Actor

To build complex systems starting from Business-Actors as base-components, the various actors have to be enabled to communicate with each other. S-BPM/ PASS defines the basics of the communication.

As mentioned before, this communication is based on point-to-point communication channels between specified subjects. In our approach, which has characteristics similar to multi agent systems (MAS) [8][9], the Business-Actors are not coupled together like subjects in S-BPM. For each Business-Actor, its zero or many behavior interfaces have to be elicitated.

In our context, a behavior interface resembles a communication protocol [4][5][7] (without loss-of-information handling, physical routing information etc.). It's focus is on the possible message sequences and the contents (in terms of data-types) of the different messages. These behavior interfaces – one for each different communication partner of a Business-Actor – can be extracted automatically from the state machine(s) within an actor. The state machine basically represents a formal graph, containing all information about communication (send & receive) to other Business-Actors and the possible communication sequences. The send- and receive-states contain the references to the data which is part of the appropriate messages. Furthermore, each state depicts a "partner role", meaning an alias for a suggested communication partner from the point of view of the Business-Actor.

Therefore, it is possible to algorithmically reduce the formal graph to the states relevant to the communication with one distinct communication partner (partner role). The possible message sequences can be derived from this reduced graph.

For each different communication partner (depicted by differently named partner roles within a Business-Actor), a separate reduced graph can be calculated. At the end, every Business-Actor has a number of behavior interfaces (BI) corresponding to the number of intended different communication partners. (There is a chance, that another Business-Actor can fulfill more than one partner role, according to ist own behavior interfaces.)

For practical purposes, as mentioned above, the Business-Actor also contains a separate block with communication rules and requirements. This block is needed to provide contextual information for evaluating possible message receivers at instance level. These rules can only be evaluated at runtime (because by then, the Business-Actor creates messages with concrete values within the data structures) and are intended for setting the communication in the correct context.

In a nutshell, there is no direct connection between two Business-Actors. A suitable communication partner can be found at runtime by evaluating a behavior interface, when a message is sent. The suitable communication partner must provide an inverted version of the behavior interface (and must additionally match all communication rules).

Therefore, any other Business-Actor can act as suitable communication partner at any point in time, as long as the behavior interfaces match. That means, that single "system components" (i.e.: microservices, Business-Actors) can be changed at any time, as long as their behavior interfaces remain the same.

4 THE BUSINESS-ACTOR VS. AN S-BPM-SUBJECT

The main differences between a Business-Actor and a "classic" S-BPM/ PASS subject are:

1. Loose – or better: no – coupling between Business-Actors, whereas subjects are linked together using the same communication channels.
2. No "refinements", which enables users to "half-automate" a subject, having do-states executed by a machine mixed with do-states executed by a human being. If a Business-Actor is implemented as a Service-Actor, its service layer(s) must provide a full implementation or service mapping for each do-state.
3. Focus on the user interface. Business-Actors, in contrast to subjects, offer the possibility to define the data representation (= user interfaces) for human users.

We believe that these changes in regards to the classic subjects can overcome the aforementioned issues in S-BPM real-world projects.

5 ADDRESSING THE PRACTICAL PROBLEMS IN STANDARD-S-BPM-TOOLS

In section II we described the practical issues that arise in real-world S-BPM projects, based on our own project experiences.

The first issue, the tight coupling between subjects, can be overcome by using the Business-Actor concept. In this case, there is no fixed coupling between the different actors and each of them can act as a true microservice.

The second issue are the often missing user-interface elicitation possibilities and expensive system changes in classic S-BPM-tools after the basic process elicitation took place. By defining the user-interface directly within the Business-Actor, we expect two main outcomes: First of all, the elicitators, facilitators and end-users (who are part of the process elicitation) are forced to focus on the look & feel of the desired frontend(s). In our personal experience it is anyway often easier for endusers to describe a "form" than to describe the data to work with in an abstract way. Secondly, because the user-interface is modeled within the Business-Actor, we are able to create a software that can interpret these models. Thus, if changes at user-interface level are needed, these changes are relatively simple model-changes. There is no necessity to re-develop or re-deploy any programmed parts of a software. At the same time, no un-changeable standard user-interface is generated – in our experience, most end-users do not like standard decisions for field- or element-positioning, therefore that is another pro of the Business-Actor approach.

Thirdly, in most of the S-BPM projects we analyzed during the last few years, system changes became expensive. This is due to missing overview, where service-calls or implemented system functionality is dispersed throughout the process (= within different subjects). This often led to increased change efforts or side-effects on the software side, if some implementations have been accidentally overlooked and did not work after a change has been made. Additionally, due to the characteristics of existing S-

BPM systems, the refinements often consist of extensive usage of internal API-calls to transfer data between an external service and the S-BPM system itself. With the Business-Actor approach, in this case the Service-Actor variant, these problems are overcome: The clear distinction between Business- and Service-Actor (human interaction vs. fully automated) enforces a separation of these two different interaction types. Throughout the system, it is always clear where to search for service-calls to IT-systems. Additionally, because of the mapping of data to service-calls and back, there is no need to handle calls to internal APIs.

We are of the opinion, that the Business-Actor concept is able to reduce the main issues that arise in "classic" S-BPM-projects and –systems.

6 THE ACTORSPHERE PLATFORM

To be able to prove the validity of the Business-Actor concept and it's pros, we decided to develop a software platform that supports the execution of Business-Actors and Service-Actors.

The platform should not act as a central system for orchestration, but instead should support the self-induced choreography of Business-Actors and Service-Actors as described in chapter 3. Therefore it should just offer a runtime environment for the two actor types and support them handling their communication.

The software has been named "actorsphere" because it represents the digital living environment for the actors: Providing infrastructure for "living" (i.e. execution) and communication (i.e. messaging).

Figure 3. The actorsphere as digital living environment for Business-Actors and Service-Actors.

The actorsphere was developed as a Java-based OSGI platform including a web-based userinterface.

Business-Actors and Service-Actors were defined by using one XML-file per actor to ensure independence from one actor to another and at the same time offering the possibility to simply manage different versions of actors by using standard (code-) version control systems like GIT.

A WSDL comprising the seven building blocks stated in chapter 3.1 has been developed also. For hopefully facilitating further discussion about standard formats for storing S-BPM(like) subject information, we add our full WSDL within the addendum (9) and are open for feedback and sharing file format information.

7 A FIRST CASE STUDY

To test the newly developed platform, we found a pilot project to validate the feasibility of the Business-Actor approach in a real-world case. We first describe the basic case and it's requirements and will focus on the outcome afterwards.

7.1 Requirements and setting

German insolvency legislation demands that companies, which are placed under insolvency administration, need to fulfill special requirements when trading goods and services.

One of these requirements is the obligation to have every single order of goods or services approved upfront by the insolvency administrator (who is by law personally accountable, should payment problems occur with these orders). On the other hand, the insolvency administrator is also responsible of maintaining a positive balance on the company's bank accounts, therefore every payment has also to be approved.

In most cases, insolvency administrators are lawyers, who take care of the legal implications of the insolvency. The business side, especially supervision of operations, is often being outsourced to companies or freelancers, who are specialized in such mandates.

These specialists have to work in many companies (up to 20) at the same time. Therefore, they need clear decision templates and all information across multiple companies and organisational structures in one place. This enables them to get to the best possible decisions without wasting time with information elicitation. Thus, also improving process quality and overview over open to-dos etc.

Decisions about orders or payments are often time-critical, because of the special situation, the ordering company is in. Maintaining operations and constant supply of goods enabling the insolvent company to earn some money is paramount.

The insolvency administration software which is used by law firms cannot be used in these cases, because it is not designed to work across different organizations. Mostly, even the business specialists for the insolvency administration have no access to the law firm's internal software installations.

In the second half of 2015, we were approached by one of the aforementioned business specialists for insolvencies, who was seeking our help in creating a software solution to improve his daily standard tasks.

The software solution should be web-based, in order to access it with any internet browser and no installation requirements. Because the users work in different organizations, a cloud-enabled solution makes sense to offer access to the same data across multiple organizations.

Many users, who work with the software, will need a very simple user interface. At best, it is based on the (process-)role, the user has, in order to make usage as simple as possible. Best way would be to offer only the functionality and information, a user needs at a specific point within the process.

7.2 Process Elicitation

Due to our lack of a software modeling tool within the actorsphere software platform, we used "classic" subject-oriented elicitation techniques in our interviews with the insolvency specialist.

Figure 4. Example of a process elicitation output on paper

The process elicitation, we chose to do by a very classic approach: Conducting single interviews directly with our customer.

The reason for this was his knowledge about the to-be-processes and his ability, to force the processes on the users of the insolvent companies which are his mandates.

The process knowledge explication during the interviews was done using S-BPM models (interaction diagrams and subject/ actor-behavior models), created with pen, paper, post-its etc. due to the current lack of a graphical software-based modeling-solution within the actorsphere software.

At the end of the elicitation process, we had created a bunch of flipcharts with post-it-notes depicting different state machines for the most important Business-Actors in the process.

For a better overview, we also created a subject interaction diagram (SID) using a simple powerpoint slide.

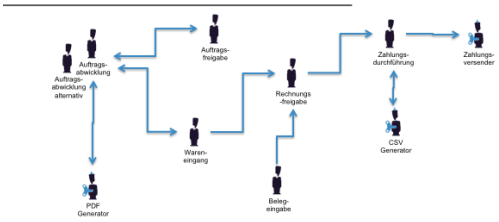


Figure 5. The first subject interaction diagram (German).

7.3 Creation of Business-Actors and Service-Actors

In the weeks following the initial process elicitation, we created the Business-Actors and Service-Actors needed for the process.

Finally, the process (better: the process network consisting of different processes) consisted of about 55 Business-Actors and Service-Actors.

We want to stress the fact, that we were able to implement every functionality used in the process network by creating actors instead of implementing features within the actorsphere.

As with many S-BPM based process automations, several refining sessions with our customer were necessary, where the customer got the possibility to do his own walkthrough through the "system" created so far and was able to give feedback, change the user interfaces etc.

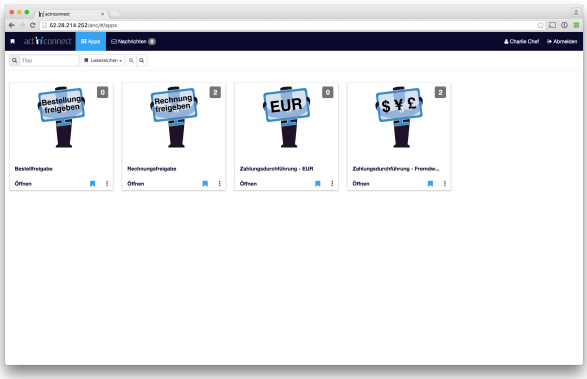


Figure 6. User interface example (I).

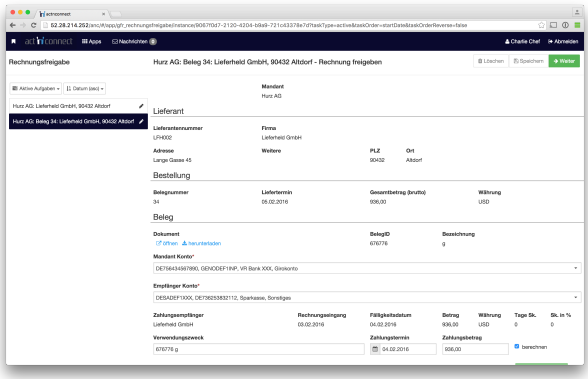


Figure 7. User interface example (II).

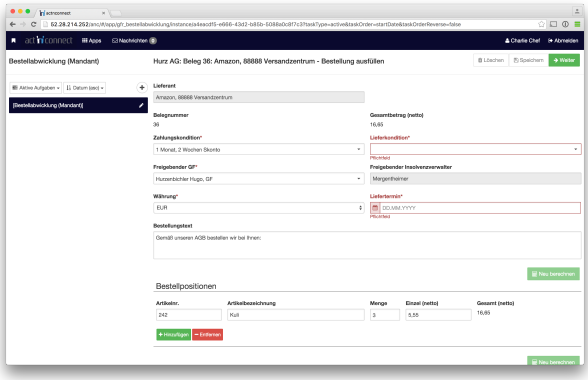


Figure 8. User interface example (III).

7.4 Lessons Learned

The case study is ongoing as of now. Even now, we are able to reflect the outcome against the main issues we identified regarding classic S-BPM in real-world projects.

Especially due to the flexibility regarding the userinterfaces and the loose coupling between different actors we created a higher identification our customer has with the software, than we are accustomed due to our past experiences.

8 CONCLUSION & FURTHER RESEARCH

Our Business-Actor concept is able to overcome the main issues that arise during real-world S-BPM projects and usage of the current S-BPM software-systems.

The loose coupling of actors, their abilities to act as fully-fledged microservices including custom user interfaces and their clear service layers seem to generate pros we want to challenge and quantify in our upcoming S-BPM projects.

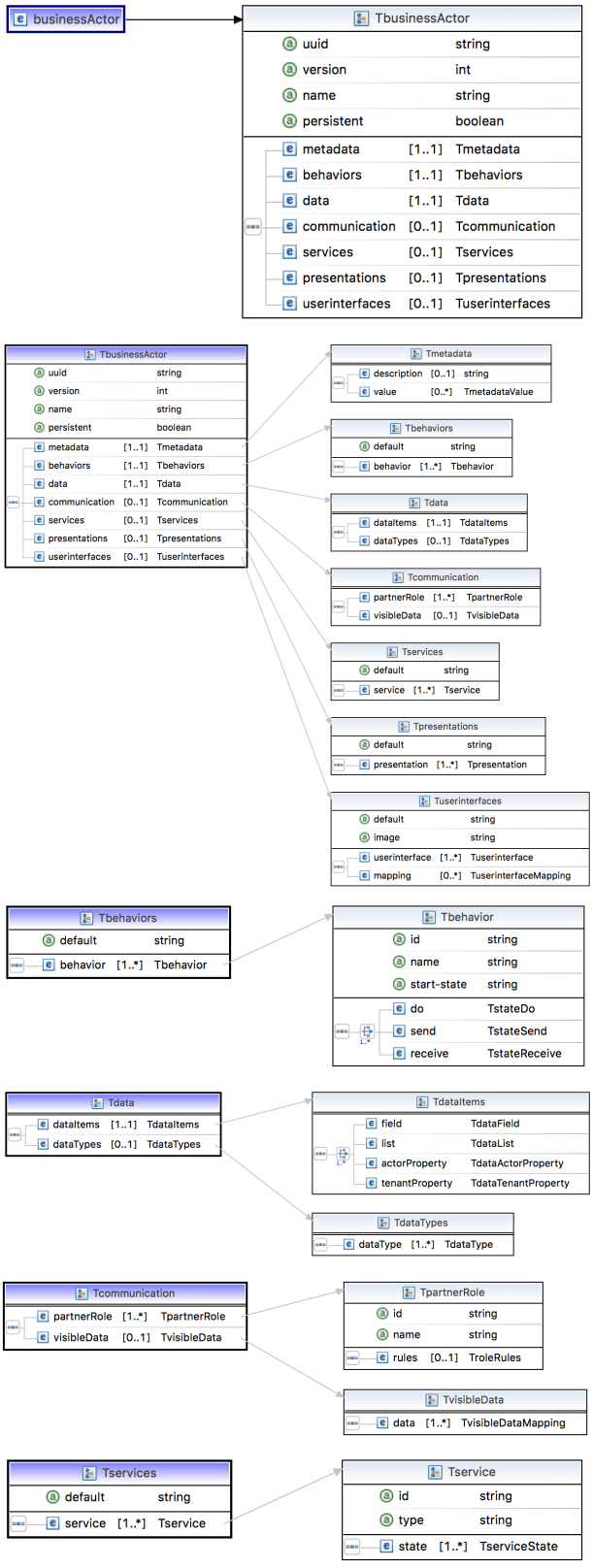
As we developed the concept based on S-BPM and loosely based on multi agent systems, we intend to deep-dive further into the foundations of multi agent systems in order to facilitate more positive effects towards S-BPM software systems.

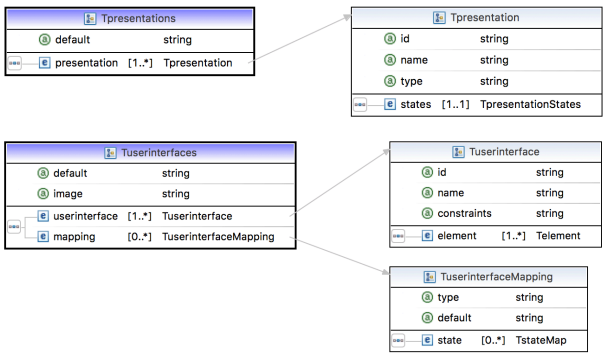
At the end, there are also some open issues within the concept, that need further research. For example, as of now, there are no formal tests, which additional criteria (if any) two Business-Actors, which can couple to more than one of each others behavior interfaces, have to fulfill preventing deadlocks.

9 ADDENDUM

9.1 WSDL

WSDL for Business-Actors used by the actorsphere software platform – due to better readability, we opted for printing a graphical representation here. The full WSDL can be retrieved at http://docs.actnconnect.com/business-actors/xsd_ba .





9.2 Example of a Service-Actor (XML)

```
<?xml version="1.0" encoding="UTF-8" ?>
<businessActor name="SA_CSVGenerator" persistent="false"
  uuid="sa_gfr_csv_generator" version="1"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://www.actnconnect.com/schema/businessactor"

xsi:schemaLocation="http://www.actnconnect.com/schema/busi
nessactor businessactor_v100.xsd ">
    <metadata>
        <description></description>
    </metadata>
    <behaviors default="standard">
        <behavior id="standard" name="Standard" start-
state="receive">
                <receive id="receive" name="Receive">
                    <segment
message="gfr_csv_files" partner-role="Requester">
      <mapping>
                                        <data
message-ref="item_list" ref="item_list" />
                                    </mapping>
                                    <transition id="create"
name="create" next-state="generateCSV"/>
                                </segment>
                </receive>

                <!-- CSV Gernator -->
                <do id="generateCSV"
name="generateCSV">
                    <data>
                                    <field readonly="false"
ref="error"/>
                                    <field readonly="false"
ref="message"/>
                                    <field readonly="false"
ref="columns"/>
                                    <field readonly="false"
ref="listItem"/>
                                    <field readonly="false"
ref="csv"/>
      <field readonly="false" ref="delimiter"/>
      <field readonly="false" ref="quoteMode"/>
      <field readonly="false" ref="escapeCharacter"/>
      <list item-add="true" item-delete="true" item-edit="true"
ref="item_list"/>
                    </data>
                    <rules>
      <transitionValidator
actor="com.actnconnect.rules.MvelTransitionValidator"
ref="next">
        <requestMapping>
          <data message-ref="expression" value="error ==
empty"/>
```

```
          <data message-ref="error" ref="error"/>
        </requestMapping>
      </transitionValidator>
      <transitionValidator
actor="com.actnconnect.rules.MvelTransitionValidator"
ref="error">
        <requestMapping>
          <data message-ref="expression" value="error !=
empty"/>
          <data message-ref="error" ref="error"/>
        </requestMapping>
      </transitionValidator>
    </rules>
                              <transitions>
                                    <transition id="next"
name="next" next-state="saveToFilesystem"/>
                                    <transition id="error"
name="next" next-state="sendError"/>
                              </transitions>
                  </do>
                  <!-- Save to filesystem -->
                  <do id="saveToFilesystem"
name="saveToFilesystem">
                              <data>
                                    <field readonly="false"
ref="error"/>
                                    <field readonly="false"
ref="message"/>
      <field readonly="false" ref="csv"/>
                                    <field readonly="false"
ref="file"/>
                                    <field readonly="false"
ref="filename"/>
                                    <field readonly="false"
ref="baInstanceId"/>
                              </data>
                              <rules>
      <transitionValidator
actor="com.actnconnect.rules.MvelTransitionValidator"
ref="next">
        <requestMapping>
          <data message-ref="expression" value="error ==
empty"/>
          <data message-ref="error" ref="error"/>
        </requestMapping>
      </transitionValidator>
      <transitionValidator
actor="com.actnconnect.rules.MvelTransitionValidator"
ref="error">
        <requestMapping>
          <data message-ref="expression" value="error !=
empty"/>
          <data message-ref="error" ref="error"/>
        </requestMapping>
      </transitionValidator>
    </rules>
                              <transitions>
                                    <transition id="next"
name="next" next-state="sendResult"/>
                                    <transition id="error"
name="error" next-state="sendError"/>
                              </transitions>

                  </do>

                  <!-- Send response -->
                  <send id="sendResult" message="csvFile"
name="sendResult" partner-role="Requester">
                              <mapping>
                                    <data message-
ref="file" ref="file"/>
                              </mapping>
                              <transition id="next"
name="next" next-state="end"/>
                  </send>
                  <send id="sendError" message="error"
name="sendError" partner-role="Requester">
                              <mapping>
                                    <data message-
ref="error" ref="error"/>
                                    <data message-
ref="message" ref="message"/>
                              </mapping>
                              <transition id="next"
name="next" next-state="end"/>
                  </send>

                  <do id="end" name="endSearchGet">
                              <data>
                                    <field readonly="true"
ref="error"/>
      <list item-add="false" item-delete="false" item-
edit="false" ref="search_items_list"/>
                              </data>
                  </do>
          </behavior>
    </behaviors>
    <data>
          <dataItems>
                  <field id="error" default-value=""
name="Error" type="string"/>
                  <field id="message" default-value=""
name="Message" type="string"/>
                  <!-- CSV -->
    <list id="item_list" name="item_list" type="items_data"/>
                  <field id="columns" default-
value="auftraggeber:Auftraggebername,aBIC:Auftraggeber
BIC,aIBAN:Auftraggeber
IBAN,zahlungsempfaenger:Zahlungsempfänger,eBIC:Empfäng
er BIC,eIBAN:Empfänger
IBAN,verwendungszweck:Verwendungszweck,zahlungsbetrag:
Betrag" name="columns" type="string"/>
                  <field id="listItem" default-
value="item_list" name="listItem" type="string"/>
                  <field id="csv" default-value=""
name="csv" type="string"/>
    <field id="delimiter" default-value=";" name="delimiter"
type="string"/>
    <field id="quoteMode" default-value="none"
name="quoteMode" type="string"/>
    <field id="escapeCharacter" default-value="'"
name="escapeCharacter" type="string"/>
    <!-- File storage -->
                  <actorProperty id="baInstanceId"
name="ActorInstanceID" property="baInstanceId"/>
                  <field id="filename" default-
value="beleg.csv" name="Filename" type="string"/>
                  <field id="file" default-value=""
name="file" type="string"/>
          </dataItems>
  <dataTypes>

    <dataType id="items_data">
      <field default-value="" id="auftraggeber"
name="Auftraggebername" type="string"/>
      <field default-value="" id="aBIC" name="Auftraggeber
BIC" type="string"/>
```

```
      <field default-value="" id="aIBAN" name="Auftraggeber IBAN" type="string"/>
      <field default-value="" id="zahlungsempfaenger" name="Zahlungsempfänger" type="string"/>
      <field default-value="" id="eBIC" name="Empfänger BIC" type="string"/>
      <field default-value="" id="eIBAN" name="Empfänger IBAN" type="string"/>
      <field default-value="" id="verwendungszweck" name="Verwendungszweck" type="string"/>
      <field default-value="" id="zahlungsbetrag" name="Betrag" type="double"/>
    </dataType>
  </dataTypes>
  </data>
  <services>
        <service id="standard" type="java">
              <state ref="generateCSV">
                    <execution>
                          <methodCall>

<class>com.actnconnect.serviceactors.csv.CSVWriter</class>

    <method>writeCSV</method>
                          </methodCall>
                    </execution>
                    <inputMapping>
                          <data ref="columns" service-ref="columns"/>
      <data ref="delimiter" service-ref="delimiter"/>
      <data ref="quoteMode" service-ref="quoteMode"/>
      <data ref="escapeCharacter" service-ref="escapeCharacter"/>
                          <data ref="listItem" service-ref="listItem"/>
      <data ref="item_list" service-ref="item_list"/>
                    </inputMapping>
                    <outputMapping>
                          <data ref="message" service-ref="errorMessage"/>
                          <data ref="error" service-ref="error"/>
                          <data ref="csv" service-ref="csv"/>
                    </outputMapping>
              </state>
              <state ref="saveToFilesystem">
                    <execution>
                          <methodCall>

    <class>com.actnconnect.filestorage.swactor.FileStorage</class>
    <method>writeFile</method>
                          </methodCall>
                    </execution>
                    <inputMapping>
                          <data ref="csv" service-ref="input"/>
                          <data ref="baInstanceId" service-ref="baInstanceId"/>
                          <data ref="filename" service-ref="filename"/>
                    </inputMapping>
                    <outputMapping>
                          <data ref="message" service-ref="errorMessage"/>
                          <data ref="error" service-ref="error"/>
                          <data ref="file" service-ref="filepath"/>
                    </outputMapping>
              </state>
        </service>
  </services>

</businessActor>
```

10 REFERENCES

[1] Fleischmann, A., Schmidt, W., Stary, C., Obermeier, St., Börger, E., Subject-oriented Business Process Management, Springer, Berlin, 2012.

[2] Fleischmann, A., Schmidt, W., Stary, C., S-BPM in the Wild, Springer, Cham, 2015.

[3] Fleischmann, A., Schmidt, W., Stary, C., Agiles Prozessmanagement mittels Subjektorientierung, in: HMD 290, 2013, pp. 64-76.

[4] Hoare, C., Communicating sequential processes, Prentice Hall, New Jersey, 1985.

[5] Holzmann, G., Design and Validation of Computer Protocols, Prentice Hall, New Jersey, 1991.

[6] Oppl, S., Subject-Oriented Elicitation of Distributed Business Process Knowledge, in: Schmidt, W. (Ed.), S-BPM ONE 2011, CCIS, Springer, Berlin, 2011, Vol. 213, pp. 16-33.

[7] Tanenbaum, A., Structured computer organization, Prentice Hall, New Jersey, 1984.

[8] Weiss, G., Multiagent Systems, A Modern Approach to Distributed Artificial Intelligence, MIT Press, London, 1999.

[9] Wooldridge, M., An introduction to multiagent systems, John Wiley & Sons, Chichester, 2002.